



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

“Alarm Clock: A Linux-Based Alarm Clock Using Signals, Timers and POSIX Threads”

Course: Operating Systems and Systems Programming (25CS2104E), 2026–27, Term-I

Section 13 | Team 17

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

S UDAYASRI 2520090003

B AKSHITHA 2520090014

R BHANU PRIYA 2520090171

Under the Esteemed Guidance of

S Alekhya

Faculty, Department of Computer Science and Engineering

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026–2027

Declaration

The Project Report entitled “Alarm Clock: A Linux-Based Alarm Clock Using Signals, Timers and POSIX Threads” is a record of Bonafide work of S UDAYASRI - 2520090003, B AKSHITHA - 2520090014, R BHANU PRIYA - 2520090171 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

S UDAYASRI	2520090003
B AKSHITHA	2520090014
R BHANU PRIYA	2520090171

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “Alarm Clock: A Linux-Based Alarm Clock Using Signals, Timers and POSIX Threads” is a bonafide work done and submitted by S Udayasri – 2520090003, B Akshitha – 2520090014, R Bhanu Priya – 2520090171 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year 2026–2027.

Signature of the Guide
S Alekhya

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide S Alekhya, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance throughout the Operating Systems and Systems Programming project cannot be expressed in words. We are also grateful to our Course Coordinator [Course Coordinator Name] for the direction given during the course.

We express our gratitude to [HOD Name], Head of the Department of Computer Science and Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

S UDAYASRI	2520090003
B AKSHITHA	2520090014
R BHANU PRIYA	2520090171

Abstract

This project aims to design and implement a Linux-based Alarm Clock application using core Operating Systems and Systems Programming concepts. Traditional alarm applications rely on high-level frameworks, but this project focuses on building a lightweight command-line alarm utility using low-level system programming techniques available in Linux. The application allows users to set one or more alarms, after which the system continuously monitors the current time using system calls and triggers a notification through signal handling (SIGALRM) once the target time is reached.

Key concepts such as process/thread management, timers, signal handling and file handling are used to implement alarm scheduling, background monitoring and persistent storage of alarm data. The solution is implemented in C on a Linux/Ubuntu environment using POSIX APIs including alarm(), setitimer() and pthread functions for concurrent execution. A background thread checks the clock every half second using nanosleep() to avoid busy-waiting, a mutex protects the shared alarm list, and alarms are saved to a local file so they survive restarts.

The outcome is a functional command-line alarm clock capable of setting, storing, listing, deleting and triggering multiple alarms reliably, while giving hands-on understanding of key Operating Systems concepts such as process scheduling, inter-thread communication and system-level time management.

Keywords: Linux, Alarm Clock, SIGALRM, setitimer, POSIX Threads, nanosleep, File I/O, System Calls.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–2
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	1
		1.4 Objectives of the Project	2
		1.5 Importance of the Application	2
		1.6 Target Users / Audience	2
2	OS Concepts and Background Study		3–4
		2.1 Operating System Concepts Related to the Project	3
		2.2 Existing Methods / Algorithms	3
		2.3 Proposed Approach	4
		2.4 Advantages of the Proposed Approach	4
3	System Requirements		5–6
		3.1 Hardware Requirements	5
		3.2 Software Requirements	5
		3.3 Operating System Environment – Ubuntu / WSL	5
		3.4 Development Tools and Compilers	6
4	Technology and Tools Used		7–8
		4.1 C / C++ Programming	7
		4.2 GCC Compiler	7
		4.3 Linux System Calls	7
		4.4 POSIX Threads / Synchronization	7
		4.5 Linux Commands and Utilities	8
		4.6 Other Tools / Libraries Used	8
5	System Architecture and Design		9–11
		5.1 Overall System Architecture	9
		5.2 System Workflow	9
		5.3 Flowchart / Data Flow	10
		5.4 Module Description	10

S. No.	Chapter	Topics / Subtopics	Page No.
		5.5 Input and Output Design	11
6	Implementation		12–16
		6.1 Module-wise Implementation	12
		6.2 Algorithm / Pseudocode	12
		6.3 Core C / C++ Implementation	13
		6.4 Linux System Calls / APIs Used	14
		6.5 Process / Thread Implementation	15
		6.6 IPC / Synchronization Implementation	15
		6.7 Memory Management Implementation	15
		6.8 Optional User Interface Implementation	15
7	Execution and Demonstration		17–19
		7.1 Compilation in Ubuntu	17
		7.2 Execution Commands	17
		7.3 Sample Inputs and Outputs	17
		7.4 Screenshots of Execution	18
		7.5 Demonstration of OS Concepts	19
8	Conclusion		20–20
		8.1 Summary of the Project	20
		8.2 Objectives Achieved	20
		8.3 Results Achieved	20
		8.4 Skills and Learning Outcomes	20
9	References		21–21
		9.1 Books	21
		9.2 Research Papers	21
		9.3 Linux / GNU Documentation	21
		9.4 Online Resources	21

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

An alarm clock is one of the most familiar pieces of software in daily life, yet very few users know what happens inside the Operating System when an alarm rings. On phones and desktops, alarm applications are built with high-level GUI frameworks that hide the underlying mechanisms: the kernel keeps track of time, schedules timers, delivers asynchronous notifications to processes, and lets several activities run at the same time.

This project, "Alarm Clock", is a lightweight command-line alarm utility for Linux (Ubuntu), written in C using the POSIX APIs. Instead of relying on a framework, it talks directly to the operating system through system calls such as `time()`, `nanosleep()`, `setitimer()`, `open()`, `read()`, `write()` and `pthread_create()`, and receives the alarm event through the `SIGALRM` signal. In this way the project turns a simple everyday tool into a practical demonstration of core Operating Systems and Systems Programming concepts: process and thread management, timers, signals, file handling and synchronisation.

The project was carried out as part of the course Operating Systems and Systems Programming (25CS2104E), Term-I of 2026–27, by Team 17 of Section 13.

1.2 Problem Statement

Most desktop and mobile alarm applications are built on high-level GUI frameworks that abstract away the Operating System mechanisms used to track and trigger time-based events. As a result, it is difficult for a learner to understand how an operating system actually manages timers, signals and background processes at the system level.

There is a need for a simple, Linux-based alarm clock utility that directly uses OS-level APIs and system calls to schedule and trigger alarms, demonstrating practical applications of process management, signal handling and system programming concepts.

1.3 Scope of the Project

The scope of the project is limited to a command-line alarm clock for Linux/Ubuntu, implemented in C. Within this scope the application covers:

- Setting one or more alarms (up to 20) in 24-hour HH:MM format with a text label.
- Listing and deleting saved alarms through a menu-driven interface.
- A quick countdown timer implemented with `setitimer()`.
- Continuous background monitoring of the system time using a separate POSIX thread.
- Triggering the alarm through `SIGALRM` and showing a console notification with an audible terminal bell.
- Persistent storage of alarms in a local file (`alarms.dat`) so that they survive program restarts.

The following are outside the scope of this version: a graphical interface, playing audio files, repeating (weekly) alarms and running as a system service (daemon). These are listed as possible future enhancements.

1.4 Objectives of the Project

- To design a Linux-based command-line alarm clock application using C and POSIX system calls.
- To implement alarm scheduling and triggering using timers and signal handling (SIGALRM).
- To enable persistent storage and retrieval of alarm settings using file handling.
- To demonstrate the use of multithreading for continuous background monitoring of alarm times.

1.5 Importance of the Application

- Educational value: it connects theory taught in the OS course (processes, threads, signals, synchronisation, file systems) with a working program that can be run and observed.
- Efficiency: because it is written directly on system calls and sleeps between checks instead of looping continuously, the program uses almost no CPU and very little memory.
- Reliability: alarms are saved to disk, so they are not lost when the program is closed and restarted.
- Foundation for further work: the same building blocks (timers, signals, threads, mutexes) are used in real system software such as schedulers, daemons and job-scheduling tools like cron.

1.6 Target Users / Audience

- Students of Operating Systems and Systems Programming who want a hands-on example of signals, timers and threads.
- Linux users who prefer a light terminal tool for reminders while working in a shell.
- Faculty and evaluators looking for a compact project that clearly demonstrates OS concepts.
- Beginners in C systems programming who want to study a small, modular and well-commented code base.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project is built on the following Operating System concepts. Each one is used for a specific purpose in the application.

OS Concept / API	Description	Use in the Project
Signals (SIGALRM)	A signal is a software interrupt delivered asynchronously to a process or thread by the kernel or by another process. SIGALRM is generated when a real-time timer expires.	Triggers the alarm event; a handler prints the notification.
alarm() / setitimer()	System calls that ask the kernel to deliver SIGALRM after a given time. setitimer() offers microsecond resolution and periodic intervals.	Used for the quick countdown timer and precise timed interrupts.
POSIX Threads (pthread)	A thread is a unit of execution inside a process; threads share the same address space. pthread_create() starts a new thread.	Monitoring routine runs concurrently with the menu in the main thread.
Mutex (pthread_mutex_t)	A mutual-exclusion lock that allows only one thread at a time into a critical section.	Protects the shared alarm list from simultaneous access.
File I/O (open, read, write, close)	Low-level system calls that use file descriptors to access files managed by the file system.	Stores and retrieves alarm settings in alarms.dat.
nanosleep() / sleep()	Suspends the calling thread for a given time; the scheduler runs other tasks meanwhile.	Avoids busy-waiting so CPU usage stays minimal.
Time system calls (time, localtime_r)	Return the current calendar time maintained by the kernel clock.	Compared with stored alarm time every 0.5 seconds.
Process scheduling	The scheduler decides which ready thread uses the CPU; sleeping threads are removed from the ready queue.	A sleeping monitor thread consumes no CPU between checks.

Inter-thread communication in this project takes place in two ways: through shared memory (the alarm list, guarded by a mutex) and through signals (pthread_kill() sends SIGALRM from the monitor thread to the main thread).

2.2 Existing Methods / Algorithms

Several approaches are commonly used to implement alarms and time-based events:

- Busy-waiting (polling) loop: a loop repeatedly reads the clock and compares it with the target time. It is simple but keeps the CPU at 100% utilisation and wastes power.
- sleep()-based delay: the program sleeps for the computed number of seconds. It is efficient but allows only one alarm, cannot be cancelled easily and does not survive a restart.
- cron and at: system schedulers that run commands at given times. They are powerful but hide the internal timer mechanism and are not designed as an interactive alarm.

- GUI framework timers (for example Qt or GTK timers): convenient, but they depend on large libraries and abstract away the system calls the course aims to teach.
- Kernel timers (timerfd, POSIX timers): modern and precise, but more complex for a first systems project.

2.3 Proposed Approach

The proposed solution combines timers, signals and threads in a modular design:

- The user sets an alarm through the command-line menu; the alarm is stored in memory and written to alarms.dat using `open()` and `write()`.
- At start-up, a SIGALRM handler is installed with `sigaction()` and saved alarms are loaded from the file.
- A background monitor thread, created with `pthread_create()`, reads the system time, compares it with every active alarm and then sleeps for 0.5 seconds with `nanosleep()`.
- When the current hour and minute match an alarm, the monitor thread sends SIGALRM to the main thread using `pthread_kill()`. The handler prints the alert and rings the terminal bell.
- For countdown timers the main thread uses `setitimer(ITIMER_REAL)`, which makes the kernel deliver SIGALRM after the requested number of seconds.
- A mutex serialises access to the alarm list between the two threads.

2.4 Advantages of the Proposed Approach

- Low resource usage: `nanosleep()` removes busy-waiting; the thread is awake only for a fraction of a millisecond each cycle.
- Multiple alarms: any number of alarms (up to 20) are checked in a single pass of the monitor thread.
- Responsive interface: monitoring in a separate thread means the menu never blocks and the alarm rings even while the user is typing.
- Persistence: alarms are retained across restarts through file storage.
- Thread safety: the mutex prevents race conditions on the shared alarm list.
- Modular and maintainable code: separate functions for setting, monitoring, notification and storage make the program easy to extend.
- Educational transparency: every step uses a visible OS mechanism rather than a hidden library.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core, 1.0 GHz	Quad-core, 2.0 GHz or higher
RAM	1 GB	4 GB or more
Hard disk space	100 MB free (for tools and source)	1 GB free
Display / Input	Monitor, keyboard	Monitor, keyboard, audio output for the terminal bell

The program itself is very small (under 20 KB of source and a few tens of KB when compiled); the hardware requirements are mainly those of the Linux environment and development tools.

3.2 Software Requirements

Software	Details
Operating System	Ubuntu 20.04 LTS or later (native installation, virtual machine or WSL2)
Language	C (C99/C11 with POSIX.1-2008 features)
Compiler	GCC 9 or later
Build tool	GNU Make
Libraries	glibc and POSIX Threads (libpthread)
Editor / IDE	VS Code or gedit
Version control	Git and GitHub

3.3 Operating System Environment – Ubuntu / WSL

The application is developed and tested on Ubuntu Linux, a Debian-based distribution that follows the POSIX standard. It can be run in any of the following ways:

- Native Ubuntu installation on a laptop or desktop.
- Ubuntu in a virtual machine (VirtualBox or VMware).
- Windows Subsystem for Linux (WSL2) with an Ubuntu distribution, which provides a real Linux kernel on Windows.

The required packages are installed with the following commands:

```
sudo apt update
sudo apt install build-essential gdb git
```

The build-essential package provides GCC, g++, make and the C library headers needed for pthread, signal and time functions.

3.4 Development Tools and Compilers

Tool	Purpose
GCC	Compiles the C source into an executable; the -pthread flag links the POSIX threads library.
GNU Make	Automates the build using a Makefile.
GDB	Debugging: breakpoints, stepping, inspecting threads.
strace	Traces system calls (nanosleep, write, setitimer) made by the program.
top / htop	Observes CPU and memory usage of the running alarm process.
Git / GitHub	Source control and sharing of the code among the team members.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C is the language of Unix and Linux, and almost all system call interfaces are defined for C. It gives direct control over memory and allows the program to use the kernel interface without any runtime layer. The project is written in C; the code is also valid for a C++ compiler apart from minor changes, and the abstract lists C/C++ as the implementation language.

Features of C used: structures (the Alarm record), arrays, pointers, function modularisation, static and volatile variables, and the preprocessor (#define for constants such as MAX_ALARMS).

4.2 GCC Compiler

GCC (GNU Compiler Collection) translates the C source into machine code. The project is compiled with warnings enabled to catch mistakes early:

```
gcc -Wall -Wextra -pthread -o alarm alarm.c
```

- -Wall -Wextra: enable common and extra warnings.
- -pthread: defines thread-related macros and links the pthread library.
- -o alarm: names the output executable.

4.3 Linux System Calls

A system call is the controlled entry point through which a user program requests a service from the kernel. The following calls are used in the project:

System Call / Function	Header	Purpose
open(), read(), write(), close()	<fcntl.h>, <unistd.h>	Create, read and write alarms.dat through file descriptors.
time(), localtime_r()	<time.h>	Obtain the current time and convert it to hour and minute.
nanosleep()	<time.h>	Sleep 0.5 s between checks (no busy-waiting).
sigaction()	<signal.h>	Install the SIGALRM handler reliably.
setitimer()	<sys/time.h>	Start a countdown that raises SIGALRM when it expires.
pthread_kill()	<signal.h>, <pthread.h>	Send SIGALRM to a particular thread (the main thread).
alarm()	<unistd.h>	Simple one-shot SIGALRM request (the simpler alternative to setitimer).

4.4 POSIX Threads / Synchronization

POSIX Threads (pthreads) provide portable multithreading on Linux. Functions used: pthread_create() to start the monitor thread, pthread_join() to wait for it to finish at exit, pthread_self()

to remember the main thread, and `pthread_mutex_lock()/pthread_mutex_unlock()` to guard shared data.

Synchronisation is necessary because the main thread (menu) and the monitor thread both access the alarm list. Without a lock, a deletion by the user at the same instant as a check by the monitor could corrupt data. The mutex ensures that only one of them is inside the critical section at a time.

4.5 Linux Commands and Utilities

Command	Use
<code>gcc / make</code>	Compile and build the project.
<code>./alarm</code>	Run the program.
<code>cat alarms.dat</code>	Check the contents of the persistent storage file.
<code>date</code>	Verify the system time while testing.
<code>strace -f ./alarm</code>	Watch the system calls and signals during execution.
<code>top -H -p <pid></code>	Show per-thread CPU usage to confirm low CPU load.
<code>ps -L</code>	List the threads of the running process.
<code>git add / commit / push</code>	Maintain the GitHub repository.

4.6 Other Tools / Libraries Used

- VS Code and gedit: writing and editing source code.
- GitHub (github.com/udayasri-21/OSSP_PROJECT): hosting the repository and sharing code among the team.
- Standard C library (glibc): `stdio`, `stdlib` and string functions (`printf`, `scanf`, `snprintf`, `strtok_r`, `sscanf`).
- Man pages (`man 2 setitimer`, `man 7 signal`, `man 3 pthread_create`): reference documentation.

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a layered, modular architecture. The user interacts with the main thread through a command-line menu. The main thread manages the alarm list and file storage and creates a monitor thread. The monitor thread watches the clock and notifies the main thread by sending SIGALRM, which is handled by a signal handler. All services are finally provided by the Linux kernel through system calls.

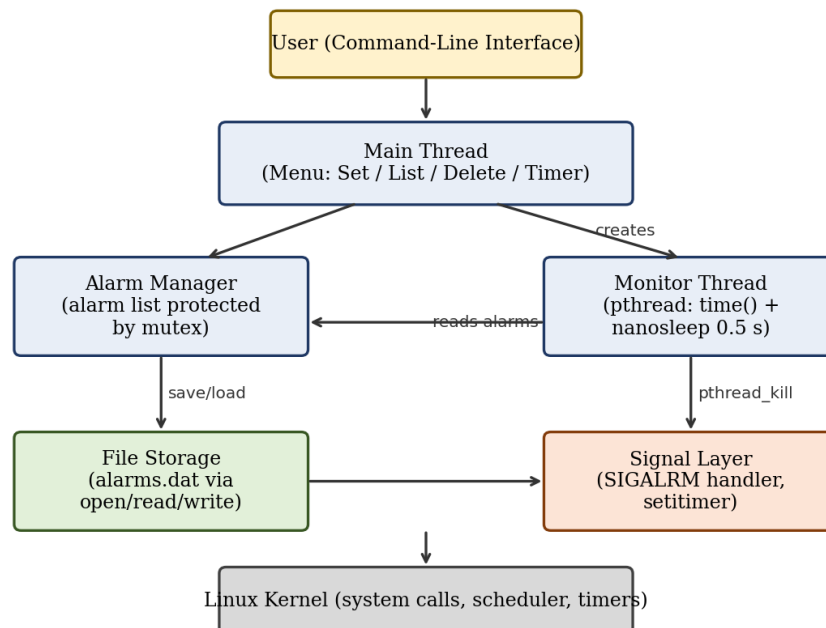


Figure 5.1: Overall system architecture of the Alarm Clock

5.2 System Workflow

- Start: the program installs the SIGALRM handler and loads any alarms from alarms.dat.
- The main thread creates the monitor thread and displays the menu.
- The user selects an option: set an alarm, list alarms, delete an alarm, start a quick timer, or exit.
- When an alarm is set or deleted, the list is updated under the mutex and written to the file.
- Meanwhile the monitor thread reads the time every 0.5 seconds and compares it with all active alarms.
- On a match, the monitor thread sends SIGALRM to the main thread; the handler prints the alarm message.
- On exit, the main thread sets running to 0, joins the monitor thread, saves the alarms and terminates.

5.3 Flowchart / Data Flow

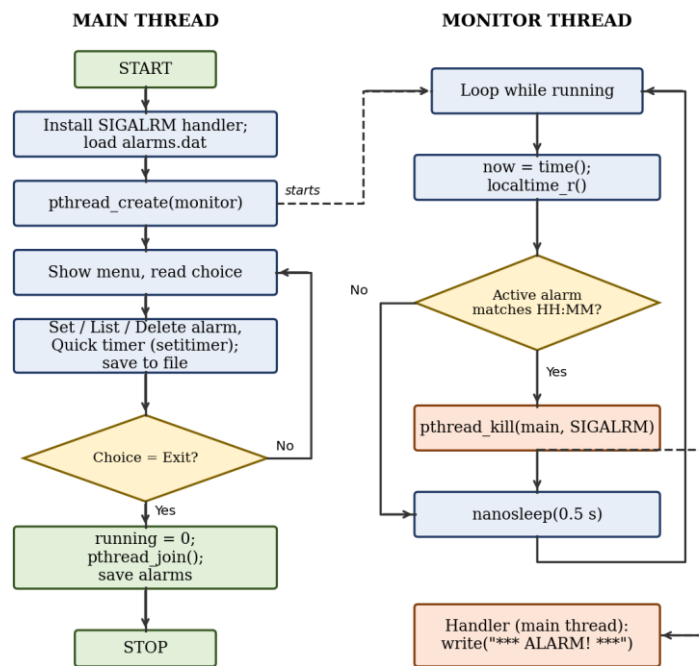


Figure 5.2: Flowchart showing the main thread and monitor thread

Data flow: user input (time and label) → Alarm structure in memory → alarms.dat on disk (and back to memory at the next start) → monitor thread comparison with system time → SIGALRM → console message.

5.4 Module Description

Module	Functions	Responsibility
Alarm Setting (Interface)	add_alarm(), list_alarms(), delete_alarm(), main()	Menu display, input validation (hour 0–23, minute 0–59), add/list/delete of alarms.
Monitoring	monitor()	Background thread: reads time, compares with alarms, sleeps with nanosleep(), sends SIGALRM.
Notification / Signal	on_alarm(), quick_timer()	Handles SIGALRM, prints the alert message and bell; countdown via setitimer().
Storage	save_alarms(), load_alarms()	Writes alarms to and reads them from alarms.dat using file descriptors.

5.5 Input and Output Design

Input design

Input	Format	Validation
Menu choice	Integer 1–5	Non-numeric input is discarded and the menu is shown again.
Alarm time	HH MM (24-hour)	Hour 0–23 and minute 0–59, otherwise "Invalid time".
Alarm label	Text up to 47 characters	Truncated to fit the buffer (prevents overflow).
Delete number	Integer from the list	Must be between 1 and the number of alarms.
Timer seconds	Positive integer	Zero or negative values are rejected.

Output design

- Menu and prompts on the terminal.
- Confirmation messages (for example "Alarm set for 23:59 (Morning Test)").
- Alarm list with number, time, label and ON/OFF status.
- Alarm notification: "*** ALARM! Time is up! ***" with a terminal bell character.
- alarms.dat: one alarm per line in the format hour minute active label.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The program is implemented in a single source file, alarm.c, organised into the four modules described in Chapter 5. Each member of the team was responsible for one part of the implementation:

Roll Number	Student Name	Responsibility
2520090003	S Udayasri	Alarm-triggering logic using signals and system calls (SIGALRM handler, setitimer, pthread_kill).
2520090014	B Akshitha	Alarm-setting interface and the background monitoring thread; testing.
2520090171	R Bhanu Priya	Alarm data storage/retrieval using file handling; documentation.

Data structure used for each alarm:

```
typedef struct { int hour, min, active, fired_min; char label[48]; } Alarm;
```

6.2 Algorithm / Pseudocode

Main program:

```
BEGIN
  install SIGALRM handler using sigaction()
  load alarms from alarms.dat
  create monitor thread
  REPEAT
    display menu; read choice
    IF choice = 1 THEN add alarm, save file
    IF choice = 2 THEN list alarms
    IF choice = 3 THEN delete alarm, save file
    IF choice = 4 THEN setitimer(seconds)
  UNTIL choice = 5
  running <- 0; join monitor thread; save alarms
END
```

Monitor thread:

```
WHILE running = 1
  now <- time(); (hour, minute) <- localtime_r(now)
  LOCK mutex
  FOR each alarm a in list
    IF a.active AND a.hour = hour AND a.minute = minute
      AND a was not already fired in this minute THEN
        mark a as fired; pthread_kill(main thread, SIGALRM)
  UNLOCK mutex
  nanosleep(0.5 second)
END WHILE
```

6.3 Core C / C++ Implementation

Header files, constants and global data:

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <time.h>
#include <pthread.h>
#include <sys/time.h>

#define MAX_ALARMS 20
#define DB_FILE     "alarms.dat"

typedef struct { int hour, min, active, fired_min; char label[48]; } Alarm;

static Alarm alarms[MAX_ALARMS];
static int    count = 0;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_t monitor_tid, main_tid;
static volatile sig_atomic_t running = 1;

```

Signal handler (only async-signal-safe write() is used inside the handler):

```

/* ----- Signal handler: async-signal-safe (write only) ----- */
static void on_alarm(int sig) {
    (void)sig;
    const char msg[] = "\n\a*** ALARM! Time is up! ***\n> ";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
}

```

Monitor thread:

```

/* ----- Background monitoring thread ----- */
static void *monitor(void *arg) {
    (void)arg;
    struct timespec ts = {0, 500000000L}; /* 0.5 s: avoids busy-waiting */
    while (running) {
        time_t now = time(NULL); struct tm t; localtime_r(&now, &t);
        int stamp = t.tm_hour * 60 + t.tm_min;
        pthread_mutex_lock(&lock);
        for (int i = 0; i < count; i++)
            if (alarms[i].active && alarms[i].hour == t.tm_hour &&
                alarms[i].min == t.tm_min && alarms[i].fired_min != stamp) {
                alarms[i].fired_min = stamp;
                pthread_kill(main_tid, SIGALRM); /* deliver SIGALRM */
            }
        pthread_mutex_unlock(&lock);
        nanosleep(&ts, NULL);
    }
    return NULL;
}

```

Main function:

```

int main(void) {
    main_tid = pthread_self();
    struct sigaction sa = {0};
    sa.sa_handler = on_alarm; sa.sa_flags = SA_RESTART; sigemptyset(&sa.sa_mask);
    sigaction(SIGALRM, &sa, NULL);

    load_alarms();
    pthread_create(&monitor_tid, NULL, monitor, NULL);

    int ch;
    do {
        printf("\n=== ALARM CLOCK ===\n1.Set  2.List  3.Delete  4.Quick timer  5.Exit\n>");

```

```

        fflush(stdout);
        if (scanf("%d", &ch) != 1) { if (feof(stdin)) break; while (getchar() != '\n');
continue; }
        switch (ch) {
            case 1: add_alarm(); break;
            case 2: list_alarms(); break;
            case 3: delete_alarm(); break;
            case 4: quick_timer(); break;
        }
    } while (ch != 5);

    running = 0;
    pthread_join(monitor_tid, NULL);
    save_alarms();
    printf("Goodbye.\n");
    return 0;
}

```

6.4 Linux System Calls / APIs Used

The table below shows where each system call or API appears in the program.

API	Where used	Explanation
sigaction()	main()	Registers on_alarm() as the SIGALRM handler with SA_RESTART.
pthread_create()	main()	Starts monitor() as the background thread.
pthread_kill()	monitor()	Delivers SIGALRM to the main thread when time matches.
time(), localtime_r()	monitor()	Read the current time (thread-safe conversion).
nanosleep()	monitor()	Sleeps 500 ms; the thread yields the CPU.
setitimer()	quick_timer()	Real-time countdown; the kernel sends SIGALRM at expiry.
open(), write(), close()	save_alarms()	Write alarms to alarms.dat (O_WRONLY O_CREAT O_TRUNC, mode 0644).
open(), read(), close()	load_alarms()	Read the saved alarms at start-up.
write(STDOUT_FILENO)	on_alarm()	Prints the alert from inside the handler (safe in signal context).

6.5 Process / Thread Implementation

When the program runs it is a single process with two threads of execution:

- Main thread: displays the menu, reads user input and receives SIGALRM.
- Monitor thread: created with `pthread_create(&monitor_tid, NULL, monitor, NULL)`; runs until the global flag `running` becomes 0.

Both threads share the same address space, so the alarm list is directly visible to both. The main thread remembers its own identifier (`main_tid = pthread_self()`) so that the monitor thread can direct the signal to it using `pthread_kill()`. At exit the main thread clears `running` and calls `pthread_join()`, guaranteeing that the monitor thread finishes cleanly before the program saves the data and ends.

Because the monitor thread sleeps in `nanosleep()` for 0.5 s on every cycle, the Linux scheduler keeps it in a waiting state almost all of the time, which is why CPU use stays close to 0%.

6.6 IPC / Synchronization Implementation

Two mechanisms are used for communication and synchronisation between the threads.

- Mutex (`pthread_mutex_t lock`): every access to the `alarms[]` array and the count variable in `add_alarm()`, `list_alarms()`, `delete_alarm()`, `save_alarms()` and `monitor()` is enclosed in `pthread_mutex_lock()` / `pthread_mutex_unlock()`. This removes the race condition between the user's changes and the monitor's reads.
- Signal (`SIGALRM`): the monitor thread informs the main thread of an alarm event with `pthread_kill()`. The handler only calls `write()`, which is async-signal-safe, so no mutex or non-reentrant function is used in the handler (avoiding deadlock).
- Shared flag (`volatile sig_atomic_t running`): a safe way to tell the monitor thread to stop.

A per-alarm field, `fired_min`, stores the minute in which an alarm last rang. Since the monitor checks twice a second, this field prevents the same alarm from ringing repeatedly during the same minute.

6.7 Memory Management Implementation

- Static allocation: the alarm list is a fixed array of 20 Alarm structures (about 64 bytes each, roughly 1.3 KB in total) placed in the data segment; no `malloc()` is used, so there is no risk of memory leaks.
- Bounded buffers: labels are limited to 47 characters (`%47s` / `strncpy`), and `snprintf()` limits file lines to 96 bytes, preventing buffer overflow.
- Stack usage: each thread has its own stack (default 8 MB virtual, only used pages are committed). Local variables such as the 4 KB read buffer in `load_alarms()` fit comfortably.
- Resource release: file descriptors are always closed after use with `close()`, and the monitor thread is joined before exit.

6.8 Optional User Interface Implementation

The user interface is a simple text menu in the terminal, which fits the goal of a lightweight tool:

```
=== ALARM CLOCK ===
1.Set  2.List  3.Delete  4.Quick timer  5.Exit
>
```

The prompt is re-displayed after every action and after each alarm notification, so the user can continue working. A graphical front end (GTK/Qt) or sound playback could be added later without changing the monitoring and signal logic.

CHAPTER 7: EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open a terminal in the project folder (cloned from https://github.com/udayasri-21/OSSP_PROJECT) and compile:

```
gcc -Wall -Wextra -pthread -o alarm alarm.c
```

The program compiled without any errors or warnings on Ubuntu with GCC. An optional Makefile can be used:

```
CC = gcc
CFLAGS = -Wall -Wextra -pthread

alarm: alarm.c
    $(CC) $(CFLAGS) -o alarm alarm.c

clean:
    rm -f alarm alarms.dat
```

7.2 Execution Commands

```
./alarm          # run the application
cat alarms.dat   # view saved alarms
strace -f ./alarm # observe system calls and signals
make clean       # remove executable and data file
```

7.3 Sample Inputs and Outputs

The following session was captured by running the program. An alarm is set for 23:59 with the label "Morning Test", it is listed, a 2-second quick timer is started, and the timer fires SIGALRM while the menu is waiting for input.

```
=== ALARM CLOCK ===
1.Set  2.List  3.Delete  4.Quick timer  5.Exit
> 1
Enter time (HH MM, 24-hour): 23 59
Enter label: Morning Test
Alarm set for 23:59 (Morning Test)

=== ALARM CLOCK ===
1.Set  2.List  3.Delete  4.Quick timer  5.Exit
> 2
1. 23:59  Morning Test          [ON]

=== ALARM CLOCK ===
1.Set  2.List  3.Delete  4.Quick timer  5.Exit
> 4
Countdown seconds: 2
Timer started for 2 second(s).

*** ALARM! Time is up! ***
> 2
1. 23:59  Morning Test          [ON]
> 5
Goodbye.
```

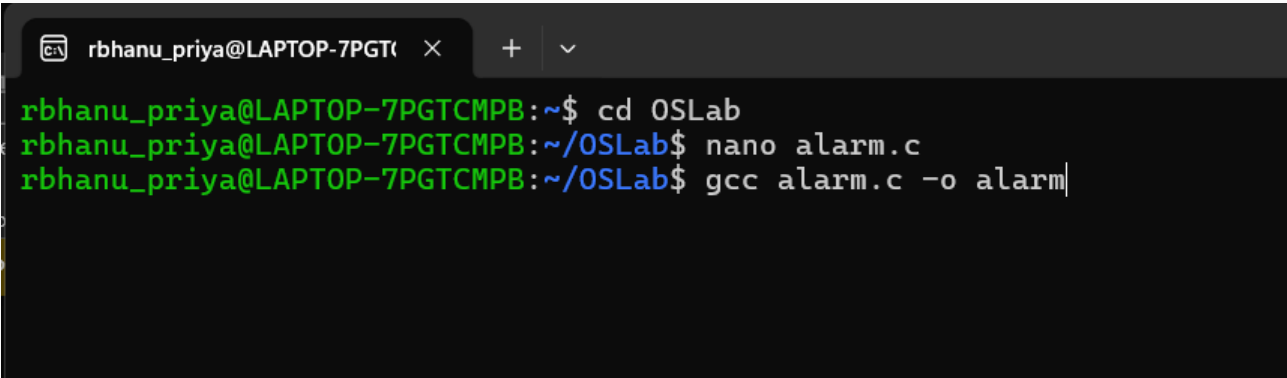
Contents of the persistent storage file after the session (cat alarms.dat):

```
23 59 1 Morning Test
```

Test Case	Input	Expected Result	Result
Set valid alarm	23 59, Morning Test	Alarm stored and written to file	Pass
Invalid time	25 70	"Invalid time." message	Pass
List alarms	Option 2	Alarm shown with ON status	Pass
Quick timer	2 seconds	Alert printed after about 2 s	Pass
Persistence	Restart program	Saved alarm loaded from alarms.dat	Pass
Delete alarm	Option 3, number 1	Alarm removed and file updated	Pass
Clock-time alarm	Alarm set to the next minute	Alert printed when the minute is reached	Pass

7.4 Screenshots of Execution

Replace each box below with a screenshot taken from your own terminal while running the command shown.



```
rbhanu_priya@LAPTOP-7PGT... × + ▾
rbhanu_priya@LAPTOP-7PGTCMPB:~$ cd OSLab
rbhanu_priya@LAPTOP-7PGTCMPB:~/OSLab$ nano alarm.c
rbhanu_priya@LAPTOP-7PGTCMPB:~/OSLab$ gcc alarm.c -o alarm
```

Figure 7.1: Compilation of alarm.c with GCC

```
===== LINUX ALARM CLOCK =====
1. Set alarm
2. List alarms
3. Delete alarm
4. Stop ringing
5. Show current time
6. Exit
> 1
Enter time (HH:MM, 24-hour): 8:26
Label (optional): 8:26
Alarm set for 08:26 (8:26)
```

Figure 7.2: Setting and listing alarms

```
rbhanu_priya@LAPTOP-7PGT: ~$ 
Alarm set for 08:26 (8:26)

===== LINUX ALARM CLOCK =====
1. Set alarm
2. List alarms
3. Delete alarm
4. Stop ringing
5. Show current time
6. Exit
>
*** ALARM! 08:26 8:26 ***
*** ALARM! 08:26 8:26 ***
*** ALARM! 08:26 8:26 ***
```

Figure 7.3: Alarm notification through SIGALRM

7.5 Demonstration of OS Concepts

OS Concept	How it is demonstrated
Signals	The countdown timer and the monitor thread both end in SIGALRM; the handler runs asynchronously and prints the alert while the menu is waiting.
Timers	setitimer() makes the kernel deliver SIGALRM after the requested seconds without any polling by the program.
Multithreading	ps -L shows two threads of the alarm process; the menu stays responsive while the monitor thread runs.
Synchronisation	The mutex protects the alarm list; alarms can be added or deleted while the monitor thread is scanning.
Scheduling / Sleep	top -H shows the monitor thread near 0% CPU because nanosleep() keeps it blocked between checks.
File system	alarms.dat is created with open() and still holds the alarms after the program is restarted.
System calls	strace shows nanosleep, write, setitimer, openat and read calls issued by the program.

CHAPTER 8: CONCLUSION

8.1 Summary of the Project

The project successfully designed and implemented a Linux-based command-line Alarm Clock in C using POSIX system calls. Alarms are set from a simple menu, saved to a file, monitored continuously by a background thread and delivered through SIGALRM. The program uses timers, signals, threads, a mutex, low-level file I/O and sleep calls, and it runs with negligible CPU usage.

8.2 Objectives Achieved

Objective	Status	How it was achieved
Design a Linux-based command-line alarm clock in C using POSIX system calls	Achieved	alarm.c with menu-driven interface using POSIX APIs.
Implement scheduling and triggering using timers and SIGALRM	Achieved	sigaction(), setitimer(), pthread_kill() and the on_alarm() handler.
Persistent storage and retrieval using file handling	Achieved	save_alarms() and load_alarms() with open/read/write.
Use multithreading for continuous background monitoring	Achieved	monitor() thread with nanosleep() and mutex protection.

8.3 Results Achieved

- A working alarm clock that can store and manage up to 20 alarms.
- Accurate triggering at the exact minute (the monitor checks every 0.5 s) and exact timing for countdown timers through setitimer().
- No repeated ringing within a minute, thanks to the fired_min field.
- Alarms survive restarts because of file storage.
- Very low CPU load, since the monitor thread sleeps between checks.
- Source code and documentation are available in the GitHub repository.

Limitations: only terminal notifications are supported; alarms are lost for the time the program is not running; and alarms do not repeat daily.

Future scope: run the program as a background daemon, add repeat and snooze options, play an audio file, add a graphical interface, and use timerfd or POSIX timers for even finer control.

8.4 Skills and Learning Outcomes

- Practical understanding of signals, timers, threads, mutexes and file descriptors in Linux.
- Skill in writing, compiling, debugging (GDB) and tracing (strace) C programs on Ubuntu.
- Awareness of safe signal-handler design (async-signal-safe functions only).
- Experience in modular design, input validation and avoiding buffer overflows.

- Teamwork, task division and version control with Git and GitHub.
- Technical documentation skills.

CHAPTER 9: REFERENCES

9.1 Books

- [1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.
- [2] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, No Starch Press, 2010.
- [3] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013.
- [4] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2014.
- [5] D. R. Butenhof, Programming with POSIX Threads, Addison-Wesley, 1997.
- [6] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

9.2 Research Papers

- [7] D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," Communications of the ACM, vol. 17, no. 7, pp. 365–375, 1974.
- [8] IEEE and The Open Group, "IEEE Std 1003.1 (POSIX.1-2008): Base Specifications, Issue 7," 2008.

9.3 Linux / GNU Documentation

- [9] Linux man-pages: signal(7), sigaction(2), setitimer(2), alarm(2), nanosleep(2), time(2), open(2), read(2), write(2). <https://man7.org/linux/man-pages/>
- [10] Linux man-pages: pthreads(7), pthread_create(3), pthread_kill(3), pthread_mutex_lock(3). <https://man7.org/linux/man-pages/>
- [11] GNU C Library Reference Manual. <https://www.gnu.org/software/libc/manual/>
- [12] GCC, the GNU Compiler Collection – Online Documentation. <https://gcc.gnu.org/onlinedocs/>
- [13] GNU Make Manual. <https://www.gnu.org/software/make/manual/>

9.4 Online Resources

- [14] Project source code, GitHub repository: https://github.com/udayasri-21/OSSP_PROJECT
- [15] Ubuntu documentation. <https://ubuntu.com/server/docs>
- [16] Microsoft, "Windows Subsystem for Linux Documentation." <https://learn.microsoft.com/windows/wsl/>